

2026 STL 기말고사 풀이 코드 — 화23수78

각 문제를 풀 때 사용한 main.cpp 전체 코드. XString.h / XString.cpp는 수정 없이 그대로 사용.

[문제 1] main.cpp

Frankenstein.txt를 vector<XString>로 읽고, 마지막 객체의 show() 호출.

main.cpp

```
//-----
// 2026          1학기          STL 기말고사          문제 1
//-----

#include "XString.h"
#include <iostream>
#include <vector>
#include <fstream>
#include <iterator>

extern bool 관찰;

//-----
int main()
//-----
{
    // [문제 1] "Frankenstein.txt" 파일에 있는 단어를 컨테이너에 저장하라.
    std::ifstream in{ "Frankenstein.txt" };
    if (not in) {
        std::cout << "Frankenstein.txt - 열 수 없습니다." << std::endl;
        return 20260617;
    }

    vector<XString> Frank{ std::istream_iterator<XString>{in}, {} };

    // Frank에 저장된 마지막 객체의 show 멤버를 호출하라.
    관찰 = true;
    Frank.back().show();
    관찰 = false;
}
```

[문제 2] main.cpp

Frank에서 가장 많이 쓰인 단어 5개를 횟수와 함께 출력. (map으로 빈도 누적 → multimap+greater로 내림차순 정렬)

main.cpp

```
//-----
// 2026          1학기          STL 기말고사          문제 2
//-----

#include "XString.h"
#include <iostream>
#include <vector>
#include <map>
#include <fstream>
#include <iterator>

extern bool 관찰;

//-----
int main()
//-----
{
    // [문제 1] "Frankenstein.txt" 파일에 있는 단어를 vector<XString>에 저장하라.
    std::ifstream in{ "Frankenstein.txt" };
    if (not in) {
        std::cout << "Frankenstein.txt - 열 수 없습니다." << std::endl;
        return 20260617;
    }

    vector<XString> Frank{ std::istream_iterator<XString>(in), {} };

    // [문제 2] Frank에 저장된 단어 중에서 가장 많이 사용된 단어를 순서대로 5개만 찾아 출력하라.
    // 단어와 사용횟수를 같이 출력하라.

    // 1. 단어별 사용 횟수를 센다. (key: 단어, value: 횟수)
    std::map<XString, size_t> 단어개수;
    for (const XString& xs : Frank)
        ++단어개수[xs];

    // 2. 횟수 기준 내림차순 정렬을 위해 key와 value를 뒤집어 multimap에 담는다.
    std::multimap<size_t, XString, std::greater<size_t>> > 개수단어;
    for (const auto& [단어, 개수] : 단어개수)
        개수단어.insert(std::make_pair(개수, 단어));

    // 3. 앞에서부터 5개만 출력한다.
    int cnt = 0;
```

```

for (const auto& [개수, 단어] : 개수단어) {
    std::cout << 단어 << " - " << 개수 << std::endl;
    if (++cnt == 5)
        break;
}

```

[문제 3] main.cpp

단어들.txt에 없는 Frank의 단어를 중복 없이 "없는 단어들.txt"에 공백 한 칸 간격으로 저장하고 개수 출력.
(vector만 사용: sort + binary_search + unique/erase)

main.cpp

```

//-----
// 2026          1학기          STL 기말고사          문제 3
//-----

#include "XString.h"
#include <iostream>
#include <vector>
#include <algorithm>
#include <fstream>
#include <iterator>

extern bool 관찰;

//-----
int main()
//-----
{
    // [문제 1] "Frankenstein.txt" 파일에 있는 단어를 vector<XString>에 저장하라.
    std::ifstream in{ "Frankenstein.txt" };
    if (not in) {
        std::cout << "Frankenstein.txt - 열 수 없습니다." << std::endl;
        return 20260617;
    }
    vector<XString> Frank{ std::istream_iterator<XString>{in}, {} };

    // [준비] "단어들.txt" 파일에 있는 단어를 words에 저장하라.
    std::ifstream in2{ "단어들.txt" };
    if (not in2) {
        std::cout << "단어들.txt - 열 수 없습니다." << std::endl;
        return 20260617;
    }
    vector<XString> words{ std::istream_iterator<XString>{in2}, {} };

```

```

// [문제 3] Frank의 단어 중 words(단어들.txt)에 존재하지 않는 단어를 찾아
//          "없는 단어들.txt"에 저장하고, 없는 단어 개수를 화면 출력하라.
// 주의) vector 이외 다른 컨테이너 사용 금지
// 주의) 결과 파일에 같은 단어가 있으면 안된다 (중복 제거)
// 주의) 단어와 단어는 공백 한 칸으로 구분

// 1. words를 이진 탐색이 가능하도록 정렬한다.
std::sort(words.begin(), words.end());

// 2. Frank에서 words에 없는 단어만 골라 없는단어 vector에 모은다.
std::vector<XString> 없는단어;
for (const XString& xs : Frank) {
    if (not std::binary_search(words.begin(), words.end(), xs))
        없는단어.push_back(xs);
}

// 3. 중복 제거를 위해 정렬 후 unique-erase 한다.
std::sort(없는단어.begin(), 없는단어.end());
auto newEnd = std::unique(없는단어.begin(), 없는단어.end());
없는단어.erase(newEnd, 없는단어.end());

// 4. "없는 단어들.txt"에 단어 사이를 공백 한 칸으로 저장한다.
std::ofstream out{ "없는 단어들.txt" };
for (size_t i = 0; i < 없는단어.size(); ++i) {
    if (i != 0)
        out << ' ';
    out << 없는단어[i];
}

// 5. 없는 단어 개수를 화면 출력한다.
std::cout << "없는 단어 개수 - " << 없는단어.size() << std::endl;

```

XString.h / XString.cpp는 시험에서 수정 지시가 없었으므로 원본 그대로 사용했습니다. PDF의 istream_iterator<YString>는 오타(클래스명은 XString)이므로 istream_iterator<XString>로 정정해 작성했습니다.